

Lightweight 3×3 Depthwise Acceleration with Sliding-Window Reuse in a Tightly Coupled RISC-V CFU for INT8 MobileNetV2

line 1: 1st Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 2nd Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 4th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 5th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 6th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

Abstract—[ABSTRACT PLACEHOLDER — to be written after Sections III–V are frozen ($\leq \sim 150$ words): problem, approach (3×3 depthwise CFU + sliding-window reuse), evaluation setup, verified headline results.]

Keywords—RISC-V, custom function unit, depthwise convolution, MobileNetV2, data reuse, FPGA, TinyML [PROPOSED]

I. INTRODUCTION

Embedded convolutional neural network (CNN) inference must fit within tight memory, latency, and energy budgets [1]. MobileNetV2 [2] was designed for such settings and replaces standard convolutions with cheaper pointwise (1×1) and depthwise (3×3) layers. Even so, one inference still involves a large amount of convolutional computation, which a small soft processor executes slowly.

A lightweight remedy is to extend the processor itself. RISC-V permits application-specific custom instructions, so a custom function unit (CFU) inside the CPU pipeline can execute selected compute-heavy operators while the embedded runtime, such as TensorFlow Lite Micro (TFLM) [3], runs the rest of the model in software. CFU Playground [4] applies this approach to TinyML on FPGAs and, for MobileNetV2, accelerates the dominant operator, 1×1 pointwise convolution.

Accelerating one operator, however, shifts the bottleneck rather than removing it. Because depthwise convolution and the other operators remain in software, the whole-model speedup reported for CFU Playground is only 3×, far below that of the accelerated operator, and depthwise convolution remains a significant part of the workload. At the other end of the design space, highly specialized accelerators fuse the complete

depthwise-separable block and achieve much higher layer-level acceleration with substantially more dedicated hardware. Less characterized is the intermediate design point: extending an existing tightly coupled CFU with depthwise support while keeping its lightweight execution model, and quantifying whether this yields a useful whole-model benefit at reasonable hardware cost.

We investigate this design point by extending the existing MobileNetV2 CFU path with a lightweight 3×3 depthwise CFU and sliding-window data reuse (Fig. 1). The depthwise CFU extends the computational coverage of the hardware; sliding-window reuse reduces redundant data movement between adjacent depthwise windows, which matters because data movement increasingly limits the performance and energy efficiency of ML hardware [5]. Our contributions are:

- A lightweight 3×3 depthwise CFU integrated alongside the existing 1×1 CFU in the RISC-V/TFLM MobileNetV2 execution path.
- Sliding-window data reuse for eligible stride-1 depthwise operations, reducing redundant input movement to the CFU.
- An end-to-end evaluation of INT8 MobileNetV2 inference covering performance, data movement, FPGA resources and timing, and functional correctness.

[FIGURE 1 PLACEHOLDER — single-column, ≈ 2.0 in]

Fig. 1. Overall system architecture: TFLM on VexRiscv, CFU with the existing 1×1 path and the new 3×3 depthwise unit, Artix-7 SoC. [PLACEHOLDER CAPTION]

II. BACKGROUND AND RELATED WORK

A. MobileNetV2 and Depthwise Convolution

Each MobileNetV2 inverted residual block applies a 1×1 expansion, a 3×3 depthwise convolution, and a 1×1 projection [2] (Fig. 2). The two layer types load hardware differently. A 1×1 convolution is a matrix–vector product at every pixel: each weight is reused across all pixels and each input across all output channels, which suits wide multiply–accumulate (MAC) units. A depthwise convolution filters each channel with its own 3×3 kernel. It needs far fewer operations, but its inputs cannot be reused across channels [6]. Its main remaining opportunity for input reuse is spatial: adjacent windows of the same channel overlap.

[FIGURE 2 PLACEHOLDER — single-column, ≈ 1.4 in]

Fig. 2. MobileNetV2 inverted residual block and operator mapping: 1×1 expansion/projection $\rightarrow$ existing 1×1 CFU; 3×3 depthwise $\rightarrow$ new 3×3 CFU; other operators $\rightarrow$ software. [PLACEHOLDER CAPTION]

B. TFLM and RISC-V CFUs

TFLM executes a quantized model by invoking one kernel per operator [3]. CFU Playground pairs TFLM with a LiteX system-on-chip built around the VexRiscv soft core on an FPGA. Its CFU follows the RISC-V R-type format, receiving two register operands and returning one result, and may keep internal state, so larger operations become short instruction sequences. Acceleration is added per operator: an optimized kernel issues custom instructions for eligible layers, while all other operators stay in software. Its MobileNetV2 CFU, the design we extend, executes 1×1 convolution with four-way INT8 MACs and hardware requantization [4].

C. Related Accelerator Approaches

Prior FPGA work falls into three groups. RISC-V/CFU-based designs keep the processor in control and add hardware incrementally; besides dense operator acceleration, RISC-V extensions have been built for sparse, pruned DNNs [7]. Specialized depthwise-separable accelerators target the block itself: a fused CFU datapath pipelines the expansion, depthwise, and projection layers and achieves high layer-level acceleration with substantially more dedicated hardware [8], and a standalone accelerator packs two 8-bit multiplications into each DSP slice [6]. Arithmetic-level work raises MAC density with mixed DSP/LUT dot-product units [9]; like sparsity, it is complementary to operator coverage.

D. Research Gap and Positioning

CFU Playground demonstrates effective 1×1 acceleration in a tightly coupled RISC-V/TFLM execution model, but its MobileNetV2 CFU leaves depthwise convolution in software; in a keyword-spotting case study, depthwise loops reuse one lane of the convolution MAC unit because a separate depthwise CFU did not fit [4]. The fused design reaches its layer-level gains with a much larger block-level datapath. This leaves an intermediate design point insufficiently characterized: a dedicated but lightweight 3×3 depthwise unit added to the existing CFU, with its whole-model performance, FPGA resource and timing cost, and data movement evaluated together. This work investigates that point and uses sliding-window reuse to address its data-movement side.

III. PROPOSED SYSTEM AND CFU ARCHITECTURE

A. Overall CPU–CFU Architecture

Fig. 1 shows the system. TFLM executes the INT8 model on a VexRiscv core. Its CONV_2D and DEPTHWISE_CONV_2D kernels are replaced by project kernels that check whether a layer is eligible for acceleration. If it is, the kernel packs operands into 32-bit words and issues custom instructions. Each instruction carries a 7-bit function code (funct7). A central dispatcher in the CFU routes the code to the matching unit.

We keep the 1×1 datapath unchanged. It handles eligible 1×1 convolutions end to end, including bias, requantization and clamping in hardware. The new 3×3 depthwise datapath sits beside it in the same CFU module and shares the same dispatcher, so it adds no new interface to the CPU.

The two datapaths differ in how much work they take over. For 1×1 layers the CFU owns the whole layer. For depthwise layers it computes only the nine-term dot product of each window. The CPU still performs window addressing, operand packing, bias, requantization, clamping and the output store. Everything else also stays in software: PAD, ADD, MEAN, FULLY_CONNECTED, SOFTMAX and the quantize/dequantize steps, the first strided 3×3 CONV_2D, and depthwise windows that overlap padding.

B. 3×3 Depthwise CFU

For one output position of one channel, the result is the sum of nine products between a 3×3 input window and that channel’s 3×3 weights:

$$y = \Sigma (x^{ij} + o^{in}) \cdot w^{ij}, i, j \in \{0, 1, 2\}$$

where o^{in} is the layer’s input offset. The raw 32-bit sum is returned to the CPU.

The CFU stores the nine input values and nine weights of the current channel in internal registers, and adds the offsets as data enters. The CPU supplies a window in three instructions: two carry four bytes of inputs and four bytes of weights each, and the third carries the ninth element. One further instruction starts the accumulation and another reads the result (Table I). The CPU stalls on the start instruction until the result is ready, so no polling is needed. Offsets are programmed once per layer.

TABLE I. 3×3 DEPTHWISE CFU INSTRUCTIONS

Instruction (funct7)	Operands	Effect
CONFIGURE (43)	input offset, weight offset	Set offsets once per layer; reset the load sequence.
LOAD (40)	4 packed inputs, 4 packed weights	Store one part of the window and weights; three calls fill the 3×3 window.
SHIFT_RIGHT (44)	one packed 3-element column	Slide the stored input window right by one column (Sec. III-D).
RUN (41)	–	Start accumulation; completes when the result is valid.
GET_RESULT (42)	–	Return the 32-bit sum.

C. Pipelined MAC Architecture

A direct implementation multiplies nine pairs and adds all nine products in one clock cycle. Our first version did this, and its adder tree dominated the critical path. The final design splits the work into two parts. First, products are computed during LOAD and stored in registers, which removes multiplication from the RUN path. Second, RUN reduces the nine stored products in three registered stages. Stage 1 adds pairs (P0+P1, P2+P3, P4+P5, P6+P7) and passes P8 through, leaving five values. Stage 2 adds pairs again, leaving three. Stage 3 forms the final sum.

No stage adds more than three numbers. Intermediate widths grow only as needed: 20-bit products, 21- and 22-bit partial sums, and a 32-bit result. A small state machine sequences the stages and signals completion when the result register is valid. RUN does not complete earlier, so a GET_RESULT issued next always reads a valid value.

D. Sliding-Window Data Reuse

At stride 1, the window for the next output position in a row overlaps the current one in two columns (Fig. 3): [A B C; D E F; G H I] becomes [B C J; E F K; H I L]. Six of the nine inputs are already inside the CFU. SHIFT_RIGHT moves the stored window one column to the right and inserts the new column (J, K, L) in a single instruction. The weights, which are fixed for a channel, are not touched.

[FIGURE 3 PLACEHOLDER — single-column, ≈ 1.8 in]

Fig. 3. 3×3 depthwise CFU datapath and sliding-window reuse. LOAD fills the input/weight windows and product registers; SHIFT_RIGHT inserts one new column (J, K, L) and retains six inputs and all weights; RUN drives the three-stage adder pipeline.

The CFU does not compute during SHIFT_RIGHT. Because the stored products no longer match the window, the next RUN first spends one extra cycle recomputing all nine products from the window registers. It then runs the same three-stage reduction. The sequences for a row are: for the first window, LOAD ×3, RUN, GET_RESULT; for each following window, SHIFT_RIGHT, RUN, GET_RESULT.

The mechanism applies only when the horizontal stride is 1, and only to output rows whose window is fully inside the input vertically. Along such a row it covers the output columns whose window is fully inside horizontally. Stride-2 layers do not overlap by two columns between consecutive windows, so they always load three words per window. Border windows that overlap padding are computed in software. In this model the stride-2 depthwise layers are preceded by explicit PAD operators, so they contain no padded windows.

E. Software/Hardware Integration

The depthwise kernel dispatches to the CFU path when all of the following hold: INT8 input and filter, a 3×3 filter, a stride of at most 2 in each dimension, dilation of 1, depth multiplier of 1, and a zero weight offset. Any other layer takes the TFLM reference path. CONFIGURE is issued once at the start of each accelerated layer.

Reuse needs the weights of one channel to stay in the CFU while the window moves along x. TFLM’s default order, $y \rightarrow x \rightarrow \text{channel}$, would change channel at every position. For eligible rows the kernel therefore iterates $y \rightarrow \text{channel} \rightarrow x$. The first window of each (row, channel) pass is loaded in full, and later windows send only the new column. The remaining positions of the row, the stride-2 rows and the border windows keep the default order. Interior windows there use three LOADs, and padded windows use a scalar loop. For every CFU window, the CPU reads the 32-bit sum and then applies bias, requantization, output offset and clamp before storing the INT8 output.

IV. EXPERIMENTAL METHODOLOGY

A. Workload and Platform

Table II lists the workload and platform. All measurements use the same INT8 MobileNetV2 a0.35 model and the same

embedded input image. Hardware figures come from Vivado 2024.1 builds of the SoC.

TABLE II. PLATFORM AND CONFIGURATIONS

Item	Value
Model	MobileNetV2 a0.35 INT8, mobilenetv2_a035_224_int8.tflite
Input	224×224×3 UIN8 embedded image (150,528 B, FNV-1a 0xbe3b0c0b)
Precision	INT8 internal; FLOAT32 output, 1000 classes
Processor / software	VexRiscv (LiteX/CFU-Playground SoC), TFLM
CFU design / tools	Amaranth HDL; Vivado 2024.1
FPGA	XC7A100T-CSG324-1 (Nexys4 DDR SoC target)
Clock	75 MHz system clock (100 MHz board clock)
Runtime platform	[PENDING: simulation or board; not stated in the logs]
C1	Existing 1×1 CFU
C2	C1 + pipelined 3×3 depthwise CFU (three LOADs per window)
C3	C2 + sliding-window reuse

B. Experimental Configurations

Three configurations form an incremental ablation that adds one capability at a time, starting from the existing 1×1 CFU system. C1 is the baseline and reference for all comparisons. C2 adds the 3×3 depthwise CFU and therefore isolates its contribution (C2 versus C1). C3 adds sliding-window reuse and therefore isolates the additional contribution of reuse (C3 versus C2). C3 versus C1 gives the total effect of the proposed additions over the existing system. All three use the same model, the same embedded image (identical input checksum in the three logs) and the same TFLM profiler. They differ only in the accelerator build. Hardware reports are taken from each configuration’s own build.

C. Evaluation Metrics

End-to-end cycles from the TFLM profiler show total inference time. Speedup is the cycles of the reference configuration divided by the cycles of the evaluated one, and cycle reduction is $1 - (\text{evaluated} / \text{reference})$. **Per-operator profiler ticks** show where the cycles go and which operators change. They are used as shares and ratios, not as absolute cycle counts. **CFU instruction counts** per inference (LOAD, SHIFT_RIGHT, RUN, GET_RESULT, CONFIGURE) are derived analytically from the layer shapes and the kernel’s counting routine. They measure data movement through the CFU interface, not time.

Instruction-count reduction, cycle reduction and speedup are three different quantities. The kernel also packs operands, reads results, requantizes and stores outputs, and SHIFT_RIGHT adds a refresh cycle to RUN. A given reduction in LOAD instructions is therefore not expected to give the same percentage reduction in cycles.

FPGA cost comes from the post-place hierarchical utilization report: LUT, FF, BRAM and DSP, for the complete design and for the CFU instance. Post-synthesis reports are not used for the final figures. **Timing** comes from the post-route

timing summary: WNS, TNS and failing endpoints against the 75 MHz clock. **Correctness** is assessed by unit-test results, top-1 class and score, and output checksum (Sec. IV-D).

D. Verification Methodology

Gateware. The CFU is verified in cycle-level Amaranth simulation by 25 unit tests. For the 3×3 unit, they check signed MAC vectors with offsets, SHIFT_RIGHT updating the window while keeping the weights, a signed sliding row against a reference, LOAD after SHIFT, and RUN latency (the SHIFT path takes one extra cycle). A system-level test drives opcodes 40–44 through the instruction interface. The 1×1 sub-blocks have their own tests.

Software/reference equivalence. The kernel can be built with per-layer checksums and instruction counters (DW3X3_VERIFY), which compare against the reference kernel. These builds change the cycle count and are not used for timing runs. [PENDING: no equivalence log is committed.]

End-to-end. C1–C3 run the same input. We compare the top-1 class and score and the printed output statistics. The kernel build also prints an output checksum (FNV-1a).

V. RESULTS AND DISCUSSION

A. End-to-End Inference Performance

TABLE III. MEASURED RESULTS (ONE RUN PER CONFIGURATION)

Metric	C1	C2	C3
Cycles	828,991,457	615,396,021	573,258,998
Speedup vs. C1	1.00×	1.347×	1.446×
Cycle reduction vs. C1	–	25.77%	30.85%
Depthwise ticks	569,466	360,818	309,533
Depthwise share	70.4%	60.1%	55.3%
LUT (total)	6,187	6,865	8,017
LUT (CFU)	1,511	1,678	2,693
FF (total)	4,504	4,898	5,080
FF (CFU)	1,081	1,475	1,657
RAMB36 / RAMB18 / DSP	15 / 33 / 8	15 / 33 / 8	15 / 33 / 8
WNS (ns)	+0.962	+0.372	+0.322
TNS (ns) / failing endpoints	0 / 0	0 / 0	0 / 0

^a Utilization is post-place and timing is post-route at 75 MHz (Vivado 2024.1). Depthwise ticks are profiler ticks.

Each added capability reduces inference cycles (Table III). The 3×3 CFU alone reduces them by 25.77% (1.347×, 213.6 M cycles). Adding window reuse reduces them by a further 6.85% relative to C2 (1.074×, 42.1 M cycles), for a total of 30.85% (1.446×) relative to C1. The profile in C1 explains why depthwise acceleration matters: depthwise layers account for 70.4% of the ticks, and the 1×1 layers were already accelerated. Speedup and cycle reduction in Table III are relative to C1.

B. Effect of the 3×3 CFU

Between C1 and C2, the depthwise ticks fall by 36.6% (569,466 to 360,818). The 1×1 CONV_2D ticks stay at about 175.6–175.9 k, as expected for an unchanged datapath. The whole-model reduction (25.77%) is smaller than the depthwise reduction because the other operators are unchanged.

Depthwise layers remain the largest operator after this change (60.1% of ticks). The CFU removes the nine multiplications and the adder tree from the CPU, but the CPU still addresses the window, packs three words, reads the result and requantizes every output.

C. Effect of Sliding-Window Reuse

Per inference, the analytical instruction counts change as follows between C2 and C3: LOAD falls from 2,529,552 to 726,864, SHIFT_RIGHT rises from 0 to 600,896, RUN and GET_RESULT stay at 843,184 each, and CONFIGURE stays at 17. LOADs fall by 71.27%, because 600,896 of the 843,184 windows (71.3%) use a SHIFT_RIGHT instead of three LOADs. The number of CFU windows does not change. Counting LOAD plus SHIFT_RIGHT, data-movement instructions fall by 47.5% (2,529,552 to 1,327,760).

Cycles fall far less, by 6.85% overall and by 14.2% in the depthwise layers (360,818 to 309,533 ticks). This is the expected difference between data movement and time. Every window still pays for RUN (including the extra refresh cycle after a shift), GET_RESULT and the software output stage. The total saving corresponds to about 70 cycles per shifted window on average (42.1 M cycles / 600,896). This is a rough figure from single runs.

Restricting the comparison to the stride-1 depthwise layers sharpens the result. The four stride-2 layers, which cannot use reuse, rose from 84,842 to 100,576 ticks between C2 and C3, although the code path they execute is unchanged. Excluding them, the other 13 depthwise layers fall from 275,976 to 208,957 ticks (−24.3%). MEAN, which is also unchanged, rose from 12,019 to 21,806 ticks. These differences point to run-to-run variation in single measurements that we cannot separate from real effects, so repeated runs are needed to quantify it.

D. FPGA Resource and Timing Results

All figures come from the post-place hierarchical utilization reports and the post-route timing summaries of each configuration’s own build. Compared with C1, the 3×3 CFU adds 167 LUTs and 394 FFs to the CFU (1,511 to 1,678 LUTs, 1,081 to 1,475 FFs). The complete design grows by 678 LUTs (+11.0%) and 394 FFs (+8.7%). Window reuse adds a further 1,015 LUTs and 182 FFs to the CFU (2,693 LUTs, 1,657 FFs).

The 3×3 unit itself grows from 198 to 1,136 LUTs and from 394 to 576 FFs. This increase is consistent with the extra window registers and shift multiplexers. It is also consistent with the multipliers used to refresh all nine products after a shift (nine, in addition to the four used during LOAD). We have not measured this attribution separately. The complete design in C3 uses 8,017 LUTs (+29.6% over C1) and 5,080 FFs (+12.8%). BRAM (15 RAMB36 and 33 RAMB18) and DSP (8, of which 4 in the CFU) are identical in all three designs, so the depthwise multipliers use LUTs only.

All three designs meet timing at 75 MHz with zero failing setup and hold endpoints (C3: 15,694 setup endpoints, WNS +0.322 ns, TNS 0, WHS +0.048 ns). WNS decreases from +0.962 ns (C1) to +0.372 ns (C2) and +0.322 ns (C3). The margin is smaller but positive. We do not report an Fmax, because it was not measured.

E. Functional Correctness

All 25 gateway unit tests pass (Ran 25 tests ... OK), including the 3×3, sliding-row and CFU-interface tests. This is simulation evidence only. In the end-to-end runs, C1, C2 and C3 process the identical input (checksum 0xbe3b0c0b) and return the same top-1 class (index 64) and score (0.406250), with the same printed output statistics. The C3 run prints an output checksum (0xd8f8f61). The C1 and C2 logs do not, so bit-exact equality of the full output vector across configurations is not demonstrated. Software equivalence tests against the reference kernel are [PENDING]. We make no claim about classification accuracy: one image was used, and the predicted label (green_mamba) is not checked against a ground-truth label.

F. Overall Discussion

The results show that a small depthwise unit added to an existing CFU is effective on the whole model. Starting from a profile dominated by depthwise layers, the 3×3 CFU removes about a quarter of the cycles for 167 CFU LUTs and no new BRAM or DSP. Reuse adds a further 6.85% for a larger LUT cost (+1,015 CFU LUTs). Per LUT, reuse is therefore less efficient than the 3×3 CFU itself, but it cuts data movement through the CFU interface by 47.5% without changing the results printed by the end-to-end runs. Both changes reduce timing slack, which stays positive at 75 MHz. The remaining cost is software work around the CFU. After C3, depthwise layers are still 55.3% of ticks, and the first strided 3×3 CONV_2D alone is about 28% of the ticks (156,491 of 559,625).

VI. LIMITATIONS AND FUTURE WORK

[SECTION VI PLACEHOLDER — not yet drafted.]

VII. CONCLUSION

[SECTION VII PLACEHOLDER — not yet drafted.]

REFERENCES

- [1] C. Banbury *et al.*, “MLPerf Tiny benchmark,” 2021, arXiv:2106.07597.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018. [VERIFY: PDF not in literature/]
- [3] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” *Proc. Mach. Learn. Syst.*, vol. 3, pp. 800–811, 2021. [VERIFY: PDF not in literature/]
- [4] S. Prakash *et al.*, “CFU Playground: Full-stack open-source framework for tiny machine learning (TinyML) acceleration on FPGAs,” 2023, arXiv:2201.01863v3. [CHECK: published version]
- [5] M. S. Vahdatpour and Y. Zhang, “Energy-efficient software–hardware co-design for machine learning: From TinyML to large language models,” 2026, arXiv:2603.23668.
- [6] X. Li, H. Huang, Y. Liu, X. Hu, and X. Xiong, “A digital signal processor-efficient accelerator for depthwise separable convolution,” *Electron. Lett.*, vol. 58, no. 7, pp. 271–273, Mar. 2022, doi: 10.1049/ell2.12435.
- [7] M. Sabih, A. Karim, J. Wittmann, F. Hannig, and J. Teich, “Hardware/software co-design of RISC-V extensions for accelerating sparse DNNs on FPGAs,” in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, 2024.
- [8] M. Yildirim and O. Ozturk, “RISC-V based TinyML accelerator for depthwise separable convolutions in edge AI,” 2025, arXiv:2511.21232.

- [9] M. Véstias, R. Duarte, J. T. de Sousa, and H. Neto, “Parallel dot-products for deep learning on FPGA.” [VERIFY: venue and year not in PDF]